

Ayurveda IPR Assistant — App, Website and Voice Interface Specification

Complete UI/UX, API Methods, Voice Architecture, File Upload, and Multilingual Speech Pipeline

Generated: August 26, 2026 | Research-backed

Table of Contents

- 1. Interface Overview: How It Works Like Claude
- 2. All Screens and UI States
- 3. Complete API Specification (Every Endpoint)
- 4. Chat Mode: Normal Text Conversation
- 5. Voice Mode: Real-Time Talk (Claude-Style)
- 6. Speak-the-Written: Read Aloud Any Response
- 7. File and Document Upload
- 8. Multilingual Language Detection and Response
- 9. Real-Time Speech: Easy vs Heavy (Research Findings)
- 10. Recommended Voice Architecture Decision
- 11. WebSocket Protocol Specification
- 12. React Native Implementation Details

1. Interface Overview: How It Works Like Claude

1.1 What Claude Does (Our Reference)

Claude has 4 distinct interaction modes. We replicate ALL of them:

MODE 1: NORMAL CHAT

User types text --> sends message --> sees "thinking" animation
--> response streams in word by word --> citations appear inline
--> response complete

MODE 2: VOICE TALK (Turn-Based)

User taps mic icon --> waveform animation shows "Listening"
--> user speaks --> natural pause detected (VAD)
--> "Processing" animation --> Claude responds with voice + text
--> user can interrupt or wait for completion

MODE 3: SPEAK THE WRITTEN (Read Aloud)

Any text response has a speaker icon
--> user taps speaker icon --> response is read aloud in TTS
--> play/pause/stop controls appear

MODE 4: FILE UPLOAD + CHAT

User taps attachment icon --> picks PDF/image/document
--> file card appears in chat input area
--> user types question about the file --> sends both
--> AI processes file (OCR if needed) + answers question

1.2 Our App: Same Pattern, Plus Domain Features

SAME AS CLAUDE:

- Normal chat with streaming responses
- Voice talk (turn-based, like Claude's voice mode)
- Speak any written response aloud
- Upload files and ask questions about them

ADDITIONAL:

- Jurisdiction toggle [India | International]
- Formulation Classification wizard
- Citation cards with expandable source text
- Confidence indicator (green/yellow/red)
- Escalation to human IP facilitator
- Multilingual: responds in the SAME language user asked in

2. All Screens and UI States

2.1 Screen Map

APP SCREENS (Expo Router file-based)	
/	Landing / Onboarding
/chat	Main Chat Interface (DEFAULT)
+--	Message list
+--	Input bar (text + mic + attach)
+--	Jurisdiction toggle
+--	Voice mode overlay (when active)
/scan	Document Scanner (camera)
/classify	Formulation Classification Wizard
/settings	Language, Profile, Data Erasure
/history	Past Conversations

2.2 Chat Screen: Every UI Element

+-----+	
Ayurveda IPR Assistant	[India v] [?]
[India International]	lang: EN
+-----+	
[Bot] Welcome! I can help with:	
- Formulation classification	
- Patent eligibility analysis	
- ABS compliance checks	
- TKDL prior art search	
How can I help you today?	
.....	
[User] Can I patent my Ashwagandha formulation?	
[Bot] Based on your query, here is my analysis:	
[Citation Card: Section 3(p), Patents Act 1970]	
An invention which in effect is traditional	
knowledge... is not patentable.	
Confidence: [GREEN - HIGH]	
[Speak] [Copy] [Share]	
This is information, not legal advice.	
[Escalate to IP Facilitator]	
+-----+	
[Attach] [Type your message...] [Mic] [Send]	
+-----+	

2.3 UI State Machine

Every interaction goes through specific states. Here is every state:

```
CHAT MESSAGE STATES:
idle          --> input bar empty, waiting for user
composing     --> user is typing text
file_attached --> file card visible in input area
sending       --> message sent, "sending" indicator
thinking      --> bot is processing (pulsing dots animation)
streaming     --> response streaming in word by word
complete      --> response fully received
error         --> something went wrong (retry button)

VOICE MODE STATES:
voice_idle    --> mic button visible, not active
voice_listening --> waveform animation, mic active
voice_processing --> "Understanding..." text, spinner
voice_responding --> bot speaking back, waveform for bot
voice_paused  --> user interrupted or bot waiting

SPEAK ALOUD STATES:
speak_idle    --> speaker icon visible on message
speak_playing --> audio playing, progress bar visible
speak_paused  --> audio paused (play/pause toggle)
speak_loading --> generating audio (first time)

FILE UPLOAD STATES:
upload_idle    --> attach button visible
upload_picking --> file picker modal open
upload_uploading --> progress bar on file card
upload_processing --> "Analyzing document..." (OCR running)
upload_ready   --> file card shows green check, ready to chat
upload_error   --> file too large or unsupported format
```

3. Complete API Specification (Every Endpoint)

3.1 REST API Endpoints

Base URL: `https://api.ayurveda-ipr.com/v1`

Authentication

Method	Endpoint	Purpose	Request Body	Response
POST	<code>/auth/register</code>	Create account	<code>{email, password, name, language_pref}</code>	<code>{user_id, token, refresh_token}</code>
POST	<code>/auth/login</code>	Login	<code>{email, password}</code>	<code>{token, refresh_token, expires_at}</code>

Method	Endpoint	Purpose	Request Body	Response
POST	/auth/refresh	Refresh JWT	{refresh_token}	{token, new_refresh_token}
POST	/auth/logout	Logout	{refresh_token}	{success: true}

Headers required on all subsequent requests:

```
Authorization: Bearer <token>
Content-Type: application/json
Accept-Language: en | hi | ta    (optional, for response language)
```

Chat (Normal Text)

Method	Endpoint	Purpose	Request Body	Response
POST	/chat/message	Send text, get AI response (streaming)	See below	SSE stream
GET	/chat/history	Get past conversations	Query: ? page=1&limit=20	{conversations: [...]}
GET	/chat/history/:conversation_id	Get specific conversation	-	{messages: [...]}
DELETE	/chat/history/:conversation_id	Delete conversation	-	{success: true}

POST /chat/message **Request Body:**

```
{
  "conversation_id": "conv_abc123",
  "message": "Can I patent my Ashwagandha formulation?",
  "jurisdiction": "india",
  "language": "en",
  "attachments": [],
  "context": {
    "previous_messages": 5
  }
}
```

POST /chat/message **Response (Server-Sent Events stream):**

```

event: start
data: {"message_id": "msg_xyz789", "status": "thinking"}

event: chunk
data: {"text": "Based on ", "citations": []}

event: chunk
data: {"text": "your query about ", "citations": []}

event: chunk
data: {"text": "Ashwagandha, ", "citations": []}

event: citation
data: {"id": "cit_001", "text": "Section 3(p), Patents Act 1970", "source": "Patents Act, 1970",
"section": "3(p)", "content_preview": "An invention which in effect..." }

event: chunk
data: {"text": "this formulation is likely barred under [Section 3(p), Patents Act 1970]. ",
"citations": ["cit_001"]}

event: metadata
data: {"confidence": "high", "confidence_score": 0.92, "jurisdiction": "india"}

event: done
data: {"message_id": "msg_xyz789", "total_tokens": 342, "citations_count": 3, "confidence": "high"}

```

File Upload and Document Processing

Method	Endpoint	Purpose	Request	Response
POST	/files/upload	Upload document	multipart/form-data: file + metadata	{file_id, status, mime_type, size}
GET	/files/:file_id/status	Check OCR processing status	-	{status: "processing" or "ready", progress: 0.75}
GET	/files/:file_id/content	Get extracted text	-	{text, pages[], metadata}
DELETE	/files/:file_id	Delete uploaded file	-	{success: true}

POST /files/upload Request (multipart/form-data):

```
Content-Type: multipart/form-data; boundary=----FormBoundary

-----FormBoundary
Content-Disposition: form-data; name="file"; filename="patent_doc.pdf"
Content-Type: application/pdf

<binary file data>
-----FormBoundary
Content-Disposition: form-data; name="metadata"
Content-Type: application/json

{
  "purpose": "chat_attachment",
  "language_hint": "hi",
  "ocr_required": true
}
-----FormBoundary--
```

POST `/files/upload` **Response:**

```
{
  "file_id": "file_abc123",
  "filename": "patent_doc.pdf",
  "mime_type": "application/pdf",
  "size_bytes": 1048576,
  "status": "processing",
  "estimated_time_seconds": 5,
  "ocr_applied": true,
  "pages_count": 12
}
```

Sending a message WITH a file attachment:

```
{
  "conversation_id": "conv_abc123",
  "message": "What ingredients are in this formulation? Is it classical or proprietary?",
  "jurisdiction": "india",
  "language": "en",
  "attachments": [
    {
      "file_id": "file_abc123",
      "type": "document"
    }
  ]
}
```

Classification

Method	Endpoint	Purpose	Request Body	Response
POST	/classify/formulation	Classify Ayurvedic product	{description, ingredients[], dosage_form}	{classification, reasoning, citations[], next_steps[]}
POST	/classify/regulatory-pathway	Determine regulatory path	{classification_result, market_target}	{pathway, requirements[], timeline}

POST /classify/formulation Request:

```
{
  "description": "A powder containing Ashwagandha root, Shatavari root, and Brahmi herb in equal parts",
  "ingredients": [
    {"name": "Ashwagandha", "botanical": "Withania somnifera", "part": "Root", "proportion": "1 part"},
    {"name": "Shatavari", "botanical": "Asparagus racemosus", "part": "Root", "proportion": "1 part"},
    {"name": "Brahmi", "botanical": "Bacopa monnieri", "part": "Whole plant", "proportion": "1 part"}
  ],
  "dosage_form": "Churna (Powder)",
  "claims": ["Rasayana", "Memory enhancement"]
}
```

Response:

```
{
  "classification": "Classical Medicine (ASU Drug)",
  "confidence": "high",
  "reasoning": "This formulation matches classical text references. All three ingredients appear in API Vol I, and similar formulations are documented in AFI Part I under Churna section.",
  "regulatory_category": "Category A - Classical Drug under D&C Act",
  "patent_eligible": false,
  "patent_bar_reason": "Section 3(p) - traditional knowledge documented in classical texts",
  "abs_required": false,
  "abs_reason": "All three plants widely cultivated. Exempt under Biodiversity Amendment 2023 for cultivated medicinal plants.",
  "citations": [
    {"text": "Section 3(p), Patents Act 1970", "content": "...traditional knowledge..."},
    {"text": "Rule 158-B, D&C Rules 1945", "content": "...classical drugs..."},
    {"text": "AFI Part I, Churna Section", "content": "..."}
  ],
  "next_steps": [
    "Apply for ASU manufacturing license under Rule 158-B",
    "No clinical trial required for classical formulation",
    "Obtain Certificate of Origin for cultivated plants"
  ]
}
```

Text-to-Speech (Speak the Written)

Method	Endpoint	Purpose	Request Body	Response
POST	/tts/synthesize	Convert text to speech audio	{text, language, voice_id}	Audio stream (PCM/MP3)
GET	/tts/voices	List available voices	Query: ? language=hi	{voices: [{id, name, language, gender, sample_url}]}

POST /tts/synthesize Request:

```
{
  "text": "Section 3(p) of the Patents Act bars patenting traditional knowledge...",
  "language": "hi",
  "voice_id": "bulbul_hi_female_01",
  "format": "mp3",
  "speed": 1.0
}
```

Response: Binary audio stream with headers:

```
Content-Type: audio/mpeg
X-Audio-Duration-Ms: 4500
X-Voice-Id: bulbul_hi_female_01
Transfer-Encoding: chunked
```

Escalation

Method	Endpoint	Purpose	Request Body	Response
POST	/escalate/request	Request human IP facilitator	{conversation_id, reason, urgency}	{ticket_id, estimated_response}
GET	/escalate/:ticket_id	Check escalation status	-	{status, facilitator_name, scheduled_time}

User Data (DPDP Compliance)

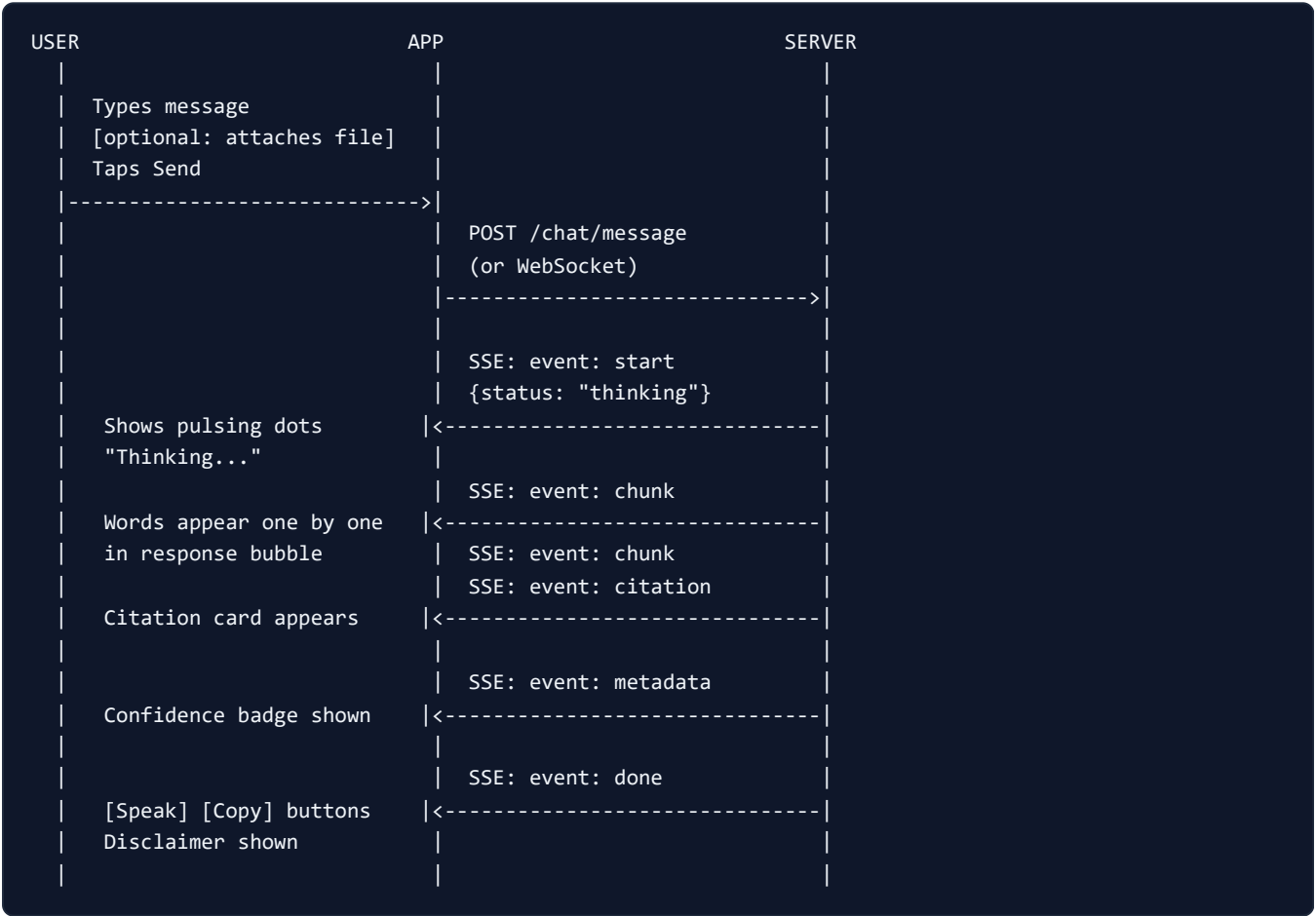
Method	Endpoint	Purpose
GET	/user/data	Export all user data
DELETE	/user/data	Erase ALL user data (right to erasure)
PATCH	/user/preferences	Update language, jurisdiction default

3.2 WebSocket Endpoints

Endpoint	Purpose	Protocol
<code>wss://api.ayurveda- ipr.com/v1/ws/voice</code>	Real-time voice conversation	Binary PCM + JSON control frames
<code>wss://api.ayurveda- ipr.com/v1/ws/chat</code>	Streaming chat (alternative to SSE)	JSON text frames

4. Chat Mode: Normal Text Conversation

4.1 Flow Diagram



4.2 React Native Implementation

```
// services/chatService.ts
export async function sendMessage(
  conversationId: string,
  message: string,
  jurisdiction: 'india' | 'international',
  language: string,
  attachments: string[],
  onChunk: (text: string) => void,
  onCitation: (citation: Citation) => void,
  onMetadata: (meta: ResponseMetadata) => void,
  onDone: (summary: DoneSummary) => void,
  onError: (error: Error) => void
) {
  const response = await fetch(`${API_BASE}/chat/message`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
      'Accept': 'text/event-stream',
    },
    body: JSON.stringify({
      conversation_id: conversationId,
      message,
      jurisdiction,
      language,
      attachments: attachments.map(id => ({ file_id: id, type: 'document' })),
    }),
  });

  const reader = response.body.getReader();
  const decoder = new TextDecoder();

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    const text = decoder.decode(value);
    const lines = text.split('\n');

    for (const line of lines) {
      if (line.startsWith('event: ')) {
        currentEvent = line.slice(7);
      } else if (line.startsWith('data: ')) {
        const data = JSON.parse(line.slice(6));
        switch (currentEvent) {
          case 'chunk': onChunk(data.text); break;
          case 'citation': onCitation(data); break;
          case 'metadata': onMetadata(data); break;
          case 'done': onDone(data); break;
        }
      }
    }
  }
}
```

5. Voice Mode: Real-Time Talk (Claude-Style)

5.1 How Claude's Voice Mode Works

Claude uses a **turn-based** approach (not full-duplex):

1. User taps mic button
2. App starts recording audio
3. Voice Activity Detection (VAD) detects speech end
4. Audio is sent to server
5. Server: ASR (speech-to-text) --> LLM (reasoning) --> TTS (text-to-speech)
6. Audio response streams back
7. App plays response audio while showing text
8. Cycle repeats

This is NOT like a phone call. There are intentional pauses for thinking. This is by design because it allows the AI to use its full reasoning capabilities (Claude uses Opus/Sonnet in voice mode, not a weaker model).

5.2 Our Voice Architecture: Identical Pattern

```
USER SPEAKS (any language: Hindi, Tamil, English, mixed)
|
v
APP: expo-audio-stream captures 16kHz mono PCM
|-- Voice Activity Detection (Silero VAD, on-device)
|-- Sends audio chunks via WebSocket (binary)
|
v
SERVER: WebSocket handler receives PCM audio
|
v
ASR ENGINE (language auto-detected):
|-- Primary: Sarvam Saaras V3 (22 Indian + English)
|-- Detects: "This is Hindi" or "This is Tamil" or "This is English"
|-- Output: transcribed text + detected_language
|
v
LANGUAGE ROUTER:
|-- If detected_language == "hi": set response_language = "hi"
|-- If detected_language == "ta": set response_language = "ta"
|-- If detected_language == "en": set response_language = "en"
|-- If code-mixed (hi+en): set response_language = "hi" (primary)
|
v
RAG PIPELINE (same as text chat):
|-- Query in detected language --> NMT to English if needed
|-- Retrieve from vector + graph + BM25
|-- LLM generates answer in English
|-- NMT translates answer back to response_language
|
v
TTS ENGINE:
|-- Primary: Sarvam Bulbul V3 (streaming WebSocket, sub-250ms)
|-- Voice selected based on response_language
|-- Streams PCM audio chunks back
|
v
APP: expo-av plays audio chunks as they arrive
|-- Text transcript also appears in chat (typed out)
|-- User can see what AI said AND hear it
```

5.3 WebSocket Voice Protocol

CLIENT --> SERVER (Control Messages - JSON text frames)

```
{
  "type": "voice_start",
  "config": {
    "sample_rate": 16000,
    "channels": 1,
    "encoding": "pcm_s16le",
    "vad_enabled": true,
    "language_hint": "hi",
    "jurisdiction": "india",
    "conversation_id": "conv_abc123"
  }
}
```

CLIENT --> SERVER (Audio Data - Binary frames)

[Raw PCM bytes, 100ms chunks = 3200 bytes each at 16kHz 16-bit mono]

CLIENT --> SERVER (Speech End)

```
{
  "type": "speech_end"
}
```

CLIENT --> SERVER (Interrupt - user wants to stop AI response)

```
{
  "type": "interrupt"
}
```

SERVER --> CLIENT (Transcription)

```
{
  "type": "transcript",
  "text": "Can I patent...",
  "language": "hi",
  "is_final": true
}
```

SERVER --> CLIENT (Status)

```
{
  "type": "status",
  "state": "thinking"
}
```

SERVER --> CLIENT (Response Text - streams alongside audio)

```
{
  "type": "response_text",
  "text": "According to ",
  "is_final": false
}
```

```
{
  "type": "response_text",
  "text": "Section 3(p)...",
  "is_final": false
}
```

SERVER --> CLIENT (Response Audio - Binary frames)

[Raw PCM audio bytes for playback]

SERVER --> CLIENT (Citation)

```
{
  "type": "citation",
  "id": "cit_001",
  "text": "Section 3(p), Patents Act 1970"
}
```

SERVER --> CLIENT (Done)

```
{
  "type": "response_done",
  "confidence": "high",
  "language": "hi"
}
```

5.4 Voice Mode UI States (What User Sees)

STATE 1: VOICE IDLE

```
+-----+
| [Mic Button - large, centered] |
| "Tap to speak"                 |
+-----+
```

STATE 2: LISTENING

```
+-----+
| [~~ Waveform Animation ~~]      |
| "Listening..."                 |
| Language detected: Hindi         |
| [Stop] button                   |
+-----+
```

STATE 3: PROCESSING

```
+-----+
| [Pulsing Circle Animation]       |
| "Understanding your question..." |
| "कृपया प्रतीक्षा करें"           |
+-----+
```

STATE 4: RESPONDING

```
+-----+
| [~~ Speaker Waveform ~~]         |
| "Speaking..."                   |
| Text appears below as spoken      |
| [Stop] [Pause] buttons            |
+-----+
```

STATE 5: WAITING FOR NEXT TURN

```
+-----+
| Response text visible above       |
| [Mic Button] "Tap to continue"    |
| or "Ask another question"        |
+-----+
```

6. Speak-the-Written: Read Aloud Any Response

6.1 What This Is

Exactly like clicking the speaker icon on ChatGPT or Claude responses. The AI response that was displayed as text gets spoken aloud.

6.2 Two Approaches

APPROACH A: Server-Side TTS (Higher Quality) - User taps speaker icon on a response message - App sends text to POST `/tts/synthesize` endpoint - Server uses Sarvam Bulbul V3 or IndicParler-TTS - Audio streams back as MP3/PCM - App plays with progress bar

APPROACH B: On-Device TTS (Zero Latency, Free) - Use React Native Expo Speech API or sherpa-onnx

- Runs locally on phone, no server call - Lower quality but instant, works offline - Good for short responses

6.3 Recommended: Hybrid Strategy

```
User taps [Speak] icon on a message
|
v
Is message < 200 characters?
|-- YES --> Use ON-DEVICE TTS (instant, free)
|           expo-speech or react-native-tts
|-- NO  --> Use SERVER TTS (higher quality)
|           POST /tts/synthesize
|           Stream audio back
|
v
LANGUAGE SELECTION:
|-- Message was in Hindi --> TTS voice = Hindi
|-- Message was in Tamil --> TTS voice = Tamil
|-- Message was in English --> TTS voice = English
|-- The language is stored in message metadata
```

6.4 On-Device TTS Implementation

```
// For WEB: Use Web Speech API (built into browsers)
function speakTextWeb(text: string, language: string) {
  const utterance = new SpeechSynthesisUtterance(text);
  utterance.lang = language === 'hi' ? 'hi-IN' :
    language === 'ta' ? 'ta-IN' : 'en-US';
  utterance.rate = 0.9; // Slightly slower for legal text
  window.speechSynthesis.speak(utterance);
}

// For MOBILE: Use expo-speech
import * as Speech from 'expo-speech';

function speakTextMobile(text: string, language: string) {
  Speech.speak(text, {
    language: language === 'hi' ? 'hi-IN' :
      language === 'ta' ? 'ta-IN' : 'en-US',
    rate: 0.9,
    onDone: () => setIsSpeaking(false),
    onStopped: () => setIsSpeaking(false),
  });
}
```

6.5 Server TTS Implementation

```
async function speakTextServer(text: string, language: string) {
  const response = await fetch(`${API_BASE}/tts/synthesize`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      text,
      language,
      voice_id: getVoiceForLanguage(language),
      format: 'mp3',
      speed: 0.9
    })
  });

  // Stream audio to player
  const audioBlob = await response.blob();
  const audioUrl = URL.createObjectURL(audioBlob);

  // On web: HTML5 Audio
  const audio = new Audio(audioUrl);
  audio.play();

  // On mobile: expo-av
  // const sound = new Audio.Sound();
  // await sound.loadAsync({ uri: audioUrl });
  // await sound.playAsync();
}
```

7. File and Document Upload

7.1 Supported File Types

File Type	Max Size	Processing	Time
PDF (text)	50 MB	PyMuPDF text extraction	< 1s
PDF (scanned)	50 MB	Surya OCR + Qwen2.5-VL	3-10s
Image (JPG/PNG)	20 MB	Surya OCR + Qwen2.5-VL	2-5s
DOCX/DOC	30 MB	python-docx extraction	< 1s
TXT/MD	10 MB	Direct read	instant

7.2 Upload Flow (Step by Step)

USER	APP	SERVER
Taps [Attach] button Picks file from device	Validates file type + size Shows file card in input +-----+ patent_doc.pdf 2.3 MB PDF [x] [===>] 45% +-----+ POST /files/upload (multipart/form-data) ----->	
	{file_id, status: processing} <-----	
	Poll: GET /files/:id/status ----->	
	{status: "processing", progress: 0.6} <-----	
	{status: "ready"} <-----	
File card shows green tick		
Types: "What ingredients are in this formulation?" Taps [Send]	POST /chat/message with attachments: [file_id] ----->	
Sees streaming response with file-specific answers	SSE stream (same as normal) <-----	

7.3 How the Server Processes Uploaded Files

```
FILE ARRIVES AT SERVER
|
v
MIME TYPE CHECK
|-- PDF? --> PyMuPDF extraction attempt
|           |-- Has extractable text? --> Use text directly
|           |-- Scanned/image-only? --> Send to Surya OCR
|-- Image? --> Send directly to Surya OCR
|-- DOCX? --> python-docx extraction
|
v
TEXT EXTRACTION COMPLETE
|
v
LANGUAGE DETECTION (on extracted text)
|-- Which language is the document in?
|-- Store detected_language
|
v
DOCUMENT UNDERSTANDING (Qwen2.5-VL 7B)
|-- "What type of document is this?"
|-- "Is this a patent application, formulation, regulatory filing?"
|-- Extract structured fields
|
v
STORE IN SESSION CONTEXT
|-- file_id -> extracted_text + metadata
|-- Available for RAG queries in this conversation
|-- User can now ask questions about this file
```

8. Multilingual Language Detection and Response

8.1 Core Principle

IMPORTANT

****The AI responds in WHATEVER language the user spoke/typed in.**** If user asks in Hindi, answer comes in Hindi. If user asks in Tamil, answer comes in Tamil. If user mixes Hindi and English (Hinglish), answer comes primarily in Hindi with English technical terms preserved.

8.2 How Language Detection Works

```
USER INPUT (text or voice)
|
v
LANGUAGE DETECTION
|
|-- TEXT INPUT:
|   |-- Script detection (Devanagari? Tamil? Latin?)
|   |-- If Devanagari script --> Hindi
|   |-- If Tamil script --> Tamil
|   |-- If Latin script --> Check content (fasttext-lid)
|   |-- If mixed scripts --> Dominant script = primary language
|
|-- VOICE INPUT:
|   |-- ASR engine (Sarvam Saaras V3) auto-detects language
|   |-- Returns: {text, detected_language}
|   |-- Handles code-switching natively
|
v
SET response_language = detected_language
|
v
RAG PIPELINE:
|-- Translate query to English (if needed) for retrieval
|-- Retrieve documents (mostly English corpus)
|-- LLM generates answer in English
|-- Translate answer to response_language
|-- Keep legal terms in original language:
|   "Section 3(p)" stays as "Section 3(p)" even in Hindi
|   "पेटेंट अधिनियम" is used alongside "Patents Act"
|
v
RESPONSE in user's language + legal citations in both
```

8.3 Supported Languages (Not Just Telugu!)

Language	Script	Voice Mode	Text Mode	TTS	ASR
English	Latin	YES	YES	YES	YES
Hindi	Devanagari	YES	YES	YES	YES
Tamil	Tamil	YES	YES	YES	YES
Telugu	Telugu	YES	YES	YES	YES
Kannada	Kannada	YES	YES	YES	YES
Malayalam	Malayalam	YES	YES	YES	YES
Bengali	Bengali	YES	YES	YES	YES
Marathi	Devanagari	YES	YES	YES	YES
Gujarati	Gujarati	YES	YES	YES	YES

Language	Script	Voice Mode	Text Mode	TTS	ASR
Punjabi	Gurmukhi	YES	YES	YES	YES
Odia	Odia	YES	YES	YES	YES
Assamese	Bengali	YES	YES	YES	YES

All 22 scheduled Indian languages are supported through **Sarvam AI** (Saaras V3 for ASR, Bulbul V3 for TTS) and **Bhashini API** (fallback for any language).

8.4 Code-Switching Handling

```
User says: "Kya main Ashwagandha ki formulation patent karwa sakta hoon?"
           (Can I patent my Ashwagandha formulation? - Hinglish)

ASR Output: {
  text: "Kya main Ashwagandha ki formulation patent karwa sakta hoon?",
  detected_language: "hi",
  code_switch_detected: true,
  segments: [
    {lang: "hi", text: "Kya main"},
    {lang: "en", text: "Ashwagandha formulation patent"},
    {lang: "hi", text: "karwa sakta hoon"}
  ]
}
```

Response Language Decision: "hi" (primary language is Hindi)

LLM Response (generated in Hindi with English legal terms):
 "Section 3(p), पेटेंट अधिनियम 1970 के अनुसार, अश्वगंधा एक पारंपरिक औषधि है जो शास्त्रीय ग्रंथों में प्रलेखित है। इसलिए, इस सूत्रीकरण को पेटेंट कराना संभव नहीं है क्योंकि यह पारंपरिक ज्ञान (traditional knowledge) की श्रेणी में आता है।

[Section 3(p), Patents Act 1970] [AFI Part I, Ashwagandhadi Churna]

विश्वास स्तर: उच्च ●
 यह जानकारी है, कानूनी सलाह नहीं।"

TTS speaks this in Hindi (Sarvam Bulbul V3, Hindi female voice)

9. Real-Time Speech: Easy vs Heavy (Research Findings)

9.1 The Big Question

Can we use lightweight, easy-to-integrate APIs for real-time speech, or do we NEED heavy server-side models?

9.2 Research Results: 3 Tiers of Solutions

TIER 1: Easy Integration APIs (Cloud, Pay-per-use)

Service	ASR	TTS	Languages	Latency	Integration	Cost
Sarvam AI	Saaras V3 (22 Indian)	Bulbul V3 (11 Indian)	22+	Sub-250ms streaming	REST + WebSocket SDK (Python, JS)	~30/hr ASR, ~15-30/10K chars TTS
Bhashini	ULCA ASR models	ULCA TTS models	22	~500ms	REST + WebSocket	FREE (government)
Deepgram	Nova-3 (multilingual)	Aura	Limited Hindi/Tamil	Sub-200ms	REST + WebSocket SDK	\$0.0043/min
ElevenLabs	Scribe	Flash v2.5	Hindi, Tamil support	Sub-300ms	REST SDK	\$0.08/1K chars
LiveKit	Mix-and-match	Mix-and-match	Any (via plugins)	Sub-400ms	React Native SDK exists	Self-hosted (free) or Cloud

Verdict: Sarvam AI is the BEST easy-integration option for Indian languages. Official Python and JavaScript SDKs, WebSocket streaming, 22 Indian languages, native code-switching. Free starting credits.

TIER 2: Self-Hosted Open Source (Server Required, Free)

Model	Type	Languages	Quality	GPU Needed	Latency
IndicConformer (AI4Bharat)	ASR	22 Indian	SOTA	1x GPU or CPU (quantized)	<100ms streaming
IndicParler-TTS (AI4Bharat)	TTS	18-23 Indian	SOTA natural	1x GPU	~500ms
Sarvam Saaras V3 (open weights)	ASR	22 Indian	SOTA	1x GPU	<100ms
faster-whisper	ASR	Multilingual	Good	1x GPU or CPU	~200ms

Verdict: This is what you use for production to avoid per-call costs. Needs a GPU server but no ongoing API fees.

TIER 3: On-Device (No Server, Runs on Phone)

Model	Type	Languages	Quality	Size	Works Offline
Sherpa-ONNX + IndicConformer	ASR	Hindi + others	Good	~188 MB	YES
Kokoro (82M)	TTS	Limited Indic	Good	~150 MB	YES

Model	Type	Languages	Quality	Size	Works Offline
Piper TTS	TTS	Limited	Decent	~50 MB	YES
expo-speech (native)	TTS	Hindi, Tamil	Robotic	Built-in	YES
Web Speech API	ASR+TTS	Hindi, Tamil	Decent	Built-in	NO

Verdict: On-device is great for "Speak the Written" (reading text aloud) and basic offline ASR. NOT good enough for real-time conversational voice mode.

9.3 Summary Decision Matrix

Feature	Easy API	Self-Hosted	On-Device
Real-time voice talk	Sarvam AI (BEST)	IndicConformer + IndicParler	NOT recommended
Speak written text	Sarvam Bulbul	IndicParler-TTS	expo-speech / Kokoro (OK for short)
Voice input (mic)	Sarvam Saaras	IndicConformer	Sherpa-ONNX (OK for commands)
Multilingual (22 langs)	Sarvam YES	AI4Bharat YES	Sherpa-ONNX partial
Code-switching	Sarvam native	Shunya/Pingala	NO
Offline mode	NO	NO (needs server)	YES
Cost	Pay per use	GPU server cost	FREE

10. Recommended Voice Architecture Decision

10.1 The Plan: Three-Layer Strategy

LAYER 1: ON-DEVICE (Free, Instant)

- expo-speech for "Speak the Written" (short texts)
- Silero VAD for voice activity detection
- Used for: quick TTS, offline, UI sounds

LAYER 2: EASY API (Sarvam AI / Bhashini)

- MVP and Hackathon phase
- Sarvam Saaras V3 for real-time ASR (WebSocket)
- Sarvam Bulbul V3 for high-quality TTS (WebSocket)
- Bhashini as free fallback
- Used for: real-time voice mode, all 22 languages

LAYER 3: SELF-HOSTED (Production)

- IndicConformer (ASR) on GPU server
- IndicParler-TTS on GPU server
- ZERO per-call cost after server setup
- Used for: scaling beyond API free tiers

10.2 Migration Path

HACKATHON (Week 1-2):

Voice = Bhashini API (free) + expo-speech (on-device TTS)

Cost: FREE

MVP (Week 3-7):

Voice = Sarvam AI APIs (WebSocket streaming)

Cost: ~5K INR/month (within free credits initially)

PRODUCTION (Week 8-12):

Voice = Self-hosted IndicConformer + IndicParler-TTS

Cost: GPU server only (no per-call fees)

Sarvam/Bhashini as fallback for rare languages

11. WebSocket Protocol Specification

11.1 Voice WebSocket Connection

Connection URL: wss://api.ayurveda-ipr.com/v1/ws/voice

Subprotocol: ayurveda-voice-v1

Authentication: Bearer token in first message

11.2 Full Message Sequence

CLIENT

SERVER

1. Connect WebSocket
ws = new WebSocket(url)
ws.binaryType = 'arraybuffer'
2. Authenticate
--> {"type": "auth", "token": "Bearer xxx"}
<-- {"type": "auth_ok", "session_id": "sess_123"}
3. Configure voice session
--> {"type": "voice_config", "config": {
 "sample_rate": 16000,
 "channels": 1,
 "encoding": "pcm_s16le",
 "jurisdiction": "india",
 "language_hint": "hi",
 "vad_mode": "auto",
 "conversation_id": "conv_abc123"
}}
<-- {"type": "config_ok"}
4. User starts speaking
--> {"type": "speech_start"}
5. Stream audio (binary frames, ~100ms each)
--> [binary: 3200 bytes of PCM]
--> [binary: 3200 bytes of PCM]
--> [binary: 3200 bytes of PCM]
...
6. User stops (VAD or manual)
--> {"type": "speech_end"}
7. Server acknowledges and transcribes
<-- {"type": "transcript_partial", "text": "Can I patent..."}
<-- {"type": "transcript_final", "text": "Can I patent my Ashwagandha formulation?", "language": "hi"}
8. Server thinks
<-- {"type": "status", "state": "thinking"}
9. Server streams response (text + audio interleaved)
<-- {"type": "response_text_start"}
<-- {"type": "response_chunk", "text": "Section 3(p) of "
<-- [binary: audio PCM chunk 1]
<-- {"type": "response_chunk", "text": "the Patents Act "
<-- [binary: audio PCM chunk 2]
<-- {"type": "citation", "id": "c1", "ref": "Section 3(p), Patents Act 1970"}
<-- {"type": "response_chunk", "text": "bars patenting traditional knowledge."}
<-- [binary: audio PCM chunk 3]
10. Response complete
<-- {"type": "response_done", "confidence": "high", "language": "hi", "message_id": "msg_xyz"}
11. Ready for next turn

```
(go back to step 4)
```

```
12. End session
```

```
--> {"type": "session_end"}
```

```
<-- {"type": "session_ended", "duration_seconds": 45}
```

11.3 Interrupt Handling

During step 9 (server streaming response):

CLIENT: User starts speaking again (interrupts)

```
--> {"type": "interrupt"}
```

SERVER: Immediately stops TTS audio generation

```
<-- {"type": "interrupted", "partial_response": "Section 3(p) of the Patents Act..."}
```

SERVER: Waits for new speech input

```
(go back to step 4)
```

12. React Native Implementation Details

12.1 Voice Recording Hook

```
// hooks/useVoiceRecording.ts
import { AudioModule } from 'expo-audio';

export function useVoiceRecording(wsRef: React.RefObject<WebSocket>) {
  const [state, setState] = useState<'idle' | 'listening' | 'processing' | 'responding'>('idle');
  const [transcript, setTranscript] = useState('');

  const startListening = async () => {
    setState('listening');

    // Send start signal
    wsRef.current?.send(JSON.stringify({ type: 'speech_start' }));

    // Start streaming audio
    await AudioModule.startRecordingStream(
      { sampleRate: 16000, channels: 1, encoding: 'pcm_16bit' },
      (audioBuffer: ArrayBuffer) => {
        if (wsRef.current?.readyState === WebSocket.OPEN) {
          wsRef.current.send(audioBuffer); // Binary frame
        }
      }
    );
  };

  const stopListening = async () => {
    await AudioModule.stopRecordingStream();
    wsRef.current?.send(JSON.stringify({ type: 'speech_end' }));
    setState('processing');
  };

  // Handle server messages
  useEffect(() => {
    if (!wsRef.current) return;

    wsRef.current.onmessage = (event) => {
      if (typeof event.data === 'string') {
        const msg = JSON.parse(event.data);
        switch (msg.type) {
          case 'transcript_final':
            setTranscript(msg.text);
            break;
          case 'status':
            if (msg.state === 'thinking') setState('processing');
            break;
          case 'response_text_start':
            setState('responding');
            break;
          case 'response_done':
            setState('idle');
            break;
        }
      } else {
        // Binary audio data - play it
      }
    };
  }, [wsRef]);
}
```

```
        playAudioChunk(event.data);
    }
};
}, [wsRef.current]);

return { state, transcript, startListening, stopListening };
}
```

12.2 File Upload Component

```
// components/FileUpload.tsx
import * as DocumentPicker from 'expo-document-picker';
import * as ImagePicker from 'expo-image-picker';

export function FileUploadButton({ onFileReady }: Props) {
  const [uploadState, setUploadState] = useState<'idle'|'uploading'|'processing'|'ready'|'error'>('idle');
  const [progress, setProgress] = useState(0);

  const pickFile = async () => {
    const result = await DocumentPicker.getDocumentAsync({
      type: ['application/pdf', 'image/*', 'application/msword'],
      copyToCacheDirectory: true,
    });

    if (result.canceled) return;

    const file = result.assets[0];
    setUploadState('uploading');

    // Upload
    const formData = new FormData();
    formData.append('file', {
      uri: file.uri,
      name: file.name,
      type: file.mimeType,
    } as any);
    formData.append('metadata', JSON.stringify({
      purpose: 'chat_attachment',
      ocr_required: file.mimeType?.includes('image') || true,
    }));

    const response = await fetch(`${API_BASE}/files/upload`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${token}` },
      body: formData,
    });

    const { file_id } = await response.json();
    setUploadState('processing');

    // Poll for processing completion
    const pollInterval = setInterval(async () => {
      const status = await fetch(`${API_BASE}/files/${file_id}/status`);
      const { status: fileStatus, progress: p } = await status.json();
      setProgress(p);

      if (fileStatus === 'ready') {
        clearInterval(pollInterval);
        setUploadState('ready');
        onFileReady(file_id);
      }
    }, 1000);
  };

  return (
```

```

    <TouchableOpacity onPress={pickFile}>
      {/* Render attach button and file card based on uploadState */}
    </TouchableOpacity>
  );
}

```

12.3 Speak Button Component

```

// components/SpeakButton.tsx
import * as Speech from 'expo-speech';

export function SpeakButton({ text, language }: { text: string, language: string }) {
  const [isSpeaking, setIsSpeaking] = useState(false);

  const handleSpeak = async () => {
    if (isSpeaking) {
      Speech.stop();
      setIsSpeaking(false);
      return;
    }

    if (text.length < 200) {
      // Short text: use on-device TTS (instant, free)
      setIsSpeaking(true);
      Speech.speak(text, {
        language: language === 'hi' ? 'hi-IN' :
          language === 'ta' ? 'ta-IN' : 'en-US',
        rate: 0.9,
        onDone: () => setIsSpeaking(false),
      });
    } else {
      // Long text: use server TTS (higher quality)
      setIsSpeaking(true);
      const audio = await fetchServerTTS(text, language);
      await playAudio(audio);
      setIsSpeaking(false);
    }
  };

  return (
    <TouchableOpacity onPress={handleSpeak}>
      <Icon name={isSpeaking ? 'stop-circle' : 'volume-2'} />
    </TouchableOpacity>
  );
}

```

Summary: Complete Feature Matrix

Feature	How It Works	API Call	Easy or Heavy?
Normal chat	Type text, get streaming response	POST <code>/chat/message</code> (SSE)	Easy (REST)

Feature	How It Works	API Call	Easy or Heavy?
Voice talk	Mic capture, real-time ASR-LLM-TTS	WSS <code>/ws/voice</code> (binary PCM)	Medium (WebSocket + Sarvam API)
Speak written	Tap speaker icon on any message	POST <code>/tts/synthesize</code> or on-device	Easy (expo-speech for short, API for long)
File upload	Attach PDF/image, ask questions	POST <code>/files/upload</code> then POST <code>/chat/message</code> with attachment	Easy (REST + polling)
Document scan	Camera capture, auto-OCR	expo-camera then POST <code>/files/upload</code>	Easy (camera + REST)
Classification	Multi-step wizard	POST <code>/classify/formulation</code>	Easy (REST)
Language switch	Auto-detect from input	Automatic (ASR detects, LLM responds same)	Sarvam handles natively
Escalation	Button or auto-trigger	POST <code>/escalate/request</code>	Easy (REST)

Version: 1.0 | Date: August 26, 2026